

2. REQUIREMENT ANALYSIS PHASE

2.1 Functional Requirements

Functional requirements describe the actions the application must support. The system shall allow authorized users to add, view, edit and search player records. Each player should have a stable unique identifier so that name changes or spelling differences do not create duplicate identities. Relevant profile fields may include name, playing role, batting style, bowling style and team association. Only information needed for performance tracking should be collected.

The system shall support team creation and match registration. Match information may include date, venue, format, participating teams, innings and completion status. Users shall enter batting statistics such as runs, balls faced, fours, sixes and dismissal status. Bowling data may include legal balls, maidens, runs conceded, wickets and supported extras. Fielding records may include catches, stumpings and run-outs, provided that the data model avoids duplicate counting.

The application shall calculate agreed statistics, display individual and team summaries, and allow users to filter records by player, team, date range, season or format when those filters are implemented. Users should be able to correct erroneous entries through a controlled edit workflow. Successful saves should produce confirmation, while rejected entries should provide specific validation messages.

If export is included, the system shall generate a file with descriptive column headings, consistent date formats and clearly represented missing values. A report should indicate the selected filters and number of records included. Functional requirements should be assigned identifiers and priorities, such as essential, desirable or future enhancement. Each requirement should have a corresponding test so that completion can be verified objectively.

2.2 Non-Functional Requirements

Non-functional requirements describe how well the application should operate. Usability requires clear navigation, readable labels, consistent forms and understandable feedback. Score-entry screens should group related information and minimize unnecessary steps.

Numeric fields should explain the accepted format. For bowling overs, the interface should make clear whether the user enters legal balls or overs-and-balls notation.

Reliability requires saved records to remain available after the application closes and restarts. Data integrity requires valid relationships between players, teams, matches and innings. The application should avoid partial updates when multiple related records are saved. Performance requirements should reflect the expected number of players and matches, and the application should remain responsive for that workload.

Security requirements include limiting administrative actions, protecting credentials if authentication is used and minimizing personal information in exported reports. Maintainability is supported by separating user-interface code, database operations and calculation logic. Portability requires documenting the runtime environment and dependencies. If the system is used beyond a local prototype, backup, recovery and access-control procedures become important.

Accessibility should be considered through adequate contrast, readable text, keyboard support where available and status messages that do not rely on color alone. The application should also handle empty datasets gracefully. These quality requirements should be reviewed during design and testing, not treated as optional polishing after all features are complete.

2.3 Data Requirements and Statistical Definitions

The data model should represent players, teams, matches, innings and performance records using stable identifiers. A player may have many batting, bowling and field records. A match may contain multiple innings, and each inning can link to the batting and bowling sides. Foreign keys should prevent performance records from referring to nonexistent players or matches. Historical team membership should be preserved where required so that a later roster change does not rewrite past records.

Statistical definitions must be agreed before implementation. Batting average is runs scored divided by the number of times dismissed. If a player has no dismissals, the application should display an agreed unavailable value such as N/A rather than divided by zero. Batting strike rate is runs divided by balls faced, multiplied by 100. If balls faced is zero, the metric is unavailable.

Bowling economy is calculated using runs conceded and overs bowled. Cricket overs notation is not decimal notation: 4.2 represents four overs and two legal balls. Internally, storing total legal balls avoids incorrect arithmetic. Economy can then be calculated as runs conceded multiplied by six, divided by legal balls. Bowling average is runs divided by wickets credited to the bowler, subject to the project's scoring rules.

The system should distinguish a legitimate zero from a missing value. For example, zero wickets is a valid result, while an empty wicket field may indicate incomplete entry. Each statistic derived should be traceable to source values. The documentation should explain how non-outs, extras and unavailable metrics are handled so users interpret reports consistently.

2.4 Constraints, Assumptions and Acceptance Criteria

Project constraints may include a limited schedule, a small development team, restricted access to historical scorecards and a requirement to run on standard computers. The first version may rely on manual score entry rather than live scoring. It is assumed that users have permission to enter the data and that scorecard values are reviewed for accuracy before being treated as final.

Acceptance criteria convert requirements into observable outcomes. A valid player can be registered and retrieved. A match can be linked to two participating teams. Performance records cannot be saved for missing players or innings.

Verified sample data produces expected batting and bowling metrics. Overs are interpreted as legal balls rather than decimal fractions. Negative values are rejected, and missing values are not silently treated as zero.

Persistence should be tested by saving records, closing the application and reopening it. Search and report filters should return only matching records. Exported data should correspond to the selected report. Error messages should identify the field or action requiring correction. These criteria provide evidence that the application works beyond a single demonstration.

Each requirement should be recorded with an ID, description, priority, source and verification method. Proposed changes should be reviewed for their effect on the database, interface, calculations and tests. This process helps maintain a manageable scope and prevents new features from introducing inconsistent data or unverified behavior.